CLaRiON : Continuous Latent Augmented-Retrieval Inference On N-cores

EL BAZ Avner, GOZE Paco

Abstract

Retrieval-Augmented Generation (RAG) architectures have recently become a central component of modern language models, allowing external document retrieval to compensate for the limited context capacity of transformer-based systems. Among these architectures, CLaRa introduced a continuous latent retrieval mechanism combining transformer encoding, differentiable top-k routing, and conditional generation. However, retrieval and inference stages remain computationally expensive, especially on CPU-based systems where embedding computation and cosine retrieval become bottlenecks.

This project presents CLaRiON, a CPU-parallel reimplementation of the CLaRa architecture designed to accelerate retrieval and inference through multi-core execution. The implementation compares a pure NumPy backend against a hybrid NumPy+Cython backend accelerated with OpenMP parallelism. The project includes encoder pretraining, joint retrieval-generation training, inference benchmarking, and profiling utilities.

Experimental results show significant performance improvements for the optimized backend, with up to $\times 2.35$ speedup during forward inference and $\times 4.95$ speedup during text generation. Training acceleration is also observed, with approximately $\times 1.68$ speedup for encoder pretraining and $\times 3.16$ speedup for joint retrieval-decoder training. These results demonstrate that CPU-oriented parallel optimization can substantially reduce inference latency in retrieval-augmented architectures while preserving comparable retrieval behavior across implementations.

1 Introduction

Retrieval-Augmented Generation (RAG) architectures have become a central paradigm for improving the factual consistency and contextual capacity of language models. Instead of relying exclusively on parametric knowledge stored in model weights, RAG systems dynamically retrieve external information and inject it into the generation process. This approach improves scalability, reduces hallucinations, and allows models to access information beyond their original training distribution. In practice, modern retrieval-based systems combine two computationally intensive stages: dense retrieval over large embedding collections and autoregressive decoding conditioned on the retrieved memory.

Retrieval-augmented language models such as RAG [Lewis et al., 2020], REALM [Guu et al., 2020], and DPR [Karpukhin et al., 2020] have established this paradigm by combining a dense retriever with a parametric generator. Recent architectures such as *CLaRa* [Apple Machine Learning Research, 2025] introduced lightweight retrieval-conditioned transformer designs where memory representations produced by an encoder are directly integrated into a decoder through cross-attention mechanisms. While effective, these pipelines remain expensive at inference time because retrieval and decoding are repeatedly executed under strict latency constraints. Dense similarity search, top-k selection, and autoregressive generation rapidly become bottlenecks, particularly when implemented with standard Python and high-level tensor abstractions. Humanity somehow managed to make

Preprint.

matrix multiplication slow by wrapping it in enough software layers. A genuine achievement in reverse engineering efficiency.

This project introduces **CLaRiON**, a CPU-parallel reimplementation inspired by the CLaRa architecture, designed to study how retrieval and generation workloads can be accelerated using optimized low-level execution strategies. The objective is not only to reproduce retrieval-augmented generation behavior, but also to analyze the trade-offs between retrieval quality, training efficiency, and inference latency under different computational backends.

CLaRiON provides multiple execution backends, including a pure NumPy implementation and an optimized NumPy+Cython backend accelerated with OpenMP parallelization. The system includes encoder pretraining, joint retrieval-decoder optimization, fused top-k retrieval kernels, autoregressive generation, and profiling utilities for performance analysis. The optimized backend significantly reduces Python overhead through contiguous memory layouts, parallel cosine similarity kernels, and fused retrieval operations.

2 Implementation

The implementation is written in Python, with performance-critical components implemented in NumPy and Cython. Rather than separating conceptual and low-level aspects, the system is designed as a unified pipeline where the encoder, retrieval mechanism, and routing logic are tightly coupled with their optimized execution backends.

The encoder maps input token sequences into dense representations suitable for retrieval-augmented generation. Each sequence $x \in \mathbb{R}^T$ is first embedded and combined with positional encodings:

$$h_0 = E(x) + P(x),$$

where embeddings and positional tables are stored as contiguous float32 arrays to ensure cache-efficient access during matrix operations.

A Transformer stack processes the sequence using residual attention blocks with RMS normalization:

$$\tilde{h}_\ell = \mathrm{RMSNorm}(h_\ell), \quad h'_\ell = h_\ell + \mathrm{Attention}(\tilde{h}_\ell), \quad h_{\ell+1} = h'_\ell + \mathrm{FFN}(\mathrm{RMSNorm}(h'_\ell)).$$

The attention mechanism follows the standard scaled dot-product formulation:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

In the NumPy backend [Harris et al., 2020], these operations are executed through vectorized BLAS calls, while in the Cython backend they are fused into a single blockwise kernel. This fused implementation processes sequences in fixed-size chunks, eliminating intermediate allocations and reducing Python overhead entirely. Attention is computed in a streaming fashion over blocks, avoiding full materialization of the $QK^\top$ matrix and reducing memory pressure for long sequences, in the same spirit as the IO-aware online-softmax recurrence used by FlashAttention [Dao et al., 2022].

Memory tokens are appended directly to the input sequence before the forward pass, forming a unified representation tensor. After the final layer, only the memory slice is retained:

$$M = h_L[-N_m :],$$

which is extracted as a contiguous view without additional copies.

To construct retrieval embeddings, memory tokens are pooled:

$$z = \mathrm{Pool}(M),$$

with either mean or first-token pooling. A learned projection then produces the query representation:

$$q = W_r z + b_r.$$

This query is compared against a precomputed memory bank

$$\mathcal{B} = \{m_1, \ldots, m_N\}, \quad m_i \in \mathbb{R}^d,$$

which is stored as a contiguous, optionally pre-normalized matrix. Similarity is computed via cosine scoring:

$$s(q, m_i) = \frac{q \cdot m_i}{\|q\|\|m_i\| \cdot \tau}.$$

In practice, normalization of the index is performed offline, reducing retrieval to a scaled dot-product operation. The dominant cost is therefore the matrix multiplication between query and memory bank, which scales as $\mathcal{O}(N \cdot d)$ but is heavily optimized through BLAS and low-level memory layouts.

Retrieval is executed through multiple backends depending on scale. The NumPy backend uses vectorized matrix multiplication for moderate workloads, while a Cython backend implements a fused similarity-and-selection kernel. This kernel computes dot products and maintains a fixed-size top-k buffer per query, eliminating the need to store full score matrices and reducing memory usage from $\mathcal{O}(B \cdot N)$ to $\mathcal{O}(B \cdot K)$. Parallelism is achieved via OpenMP-style loops over queries using `prange`, ensuring near-linear scaling with CPU cores.

Sparse routing is performed using a Top-K operator applied to score matrices:

$$\mathrm{TopK}(\mathbf{S}) = (\mathbf{I}_K, \mathbf{V}_K), \quad \mathbf{S} \in \mathbb{R}^{B \times N}.$$

In the NumPy implementation, this is handled using `argpartition`, which avoids full sorting and reduces complexity to approximately $\mathcal{O}(N)$ per row. The Cython implementation replaces this with an in-place streaming selection algorithm operating directly on contiguous buffers, further reducing overhead and improving cache locality.

The forward pass produces a binary selection mask:

$$h_{b,i} = \begin{cases} 1 & \text{if } i \in \mathrm{TopK}(\mathbf{s}_b) \\ 0 & \text{otherwise.} \end{cases}$$

To enable gradient propagation through this discrete operation, a straight-through estimator is used [Jang et al., 2016]. The forward pass remains hard, while the backward pass relies on a softmax relaxation:

$$p_{b,i} = \frac{\exp(s_{b,i}/\tau)}{\sum_j \exp(s_{b,j}/\tau)},$$

leading to the gradient:

$$\nabla s_b = \frac{1}{\tau} p_b \odot \left(\nabla y_b - \langle \nabla y_b, p_b \rangle\right).$$

The decoder maps hidden states to vocabulary logits via a linear projection:

$$\mathbf{z} = \mathbf{h} W_{\mathrm{lm}}.$$

During inference, Top-K filtering can optionally restrict sampling to a subset of tokens, renormalizing probability mass over the selected set to stabilize generation while preserving stochasticity.

When memory augmentation is enabled, decoder hidden states interact with an external memory tensor:

$$\mathbf{M} \in \mathbb{R}^{B \times S \times D},$$

through standard cross-attention:

$$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

The resulting context vectors are fused back into the hidden stream before projection to logits, ensuring that generation is conditioned jointly on autoregressive history and retrieved external memory.

Overall, the system integrates dense representation learning, approximate nearest-neighbor retrieval, sparse Top-K routing, and memory-conditioned decoding into a single pipeline. Efficiency is achieved through contiguous memory layouts, fused Cython kernels, and blockwise computation strategies, while differentiability is preserved only in routing components via straight-through estimation.

3 Parallelization Strategy

The Cython backend is built around three principles: minimise Python overhead, maximise cache locality, and parallelise on the most natural data axis.

Loop axis selection. OpenMP [Dagum and Menon, 1998] parallelism is exposed through Cython's [Behnel et al., 2011] `prange` directive. Two axes dominate the workload and are parallelised differently. The encoder iterates in parallel over the *batch* dimension, since each sequence executes the same transformer kernel independently. The retriever iterates in parallel over the *corpus* dimension during similarity scoring, which exposes a much larger amount of work than the batch axis at retrieval time (typically $N \in [10^3, 10^5]$ versus $B \leq 64$).

Fused kernels. Operations that would otherwise allocate intermediate tensors are fused into a single `cdef nogil` function. The hybrid blockwise encoder block applied to one layer reuses one preallocated `ctx` buffer to hold the streaming attention output, then writes the FFN result directly back into the input tensor (residual addition in place). The cosine-and-top-k retriever keeps a per-query top-k array of size $\mathcal{O}(K)$ instead of materialising the full $B \times N$ score matrix.

Memory layout. All arrays are declared with `mode="c"` contiguous layout and explicit pointer aliasing so that the C compiler can promote the inner loops to SIMD when possible. Reductions inside `prange` are written as plain assignments (`acc = acc + ...`) rather than `+=`, because the latter is intercepted by Cython 3 as an OpenMP reduction over the outer loop variable and triggers a compilation error in our use case.

Streaming attention. The attention block does not materialise the full $QK^\top$ matrix. Instead it processes the key/value sequence in fixed-size blocks of length B_s (typically 64) using an online softmax accumulator $(m,\, d,\, \mathrm{ctx})$ updated with the standard rescaling identity

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{block}}), \qquad \alpha = \exp(m_{\text{old}} - m_{\text{new}}),$$

which keeps activation memory proportional to $\mathcal{O}(B \cdot L \cdot H)$ rather than $\mathcal{O}(B \cdot L^2)$.

Thread scaling. The number of OpenMP threads is configurable per backend and is decoupled from BLAS thread settings to avoid oversubscription. In particular, the matmul micro-benchmarks revealed that a hand-tuned triple-loop kernel does not beat the underlying BLAS GEMM at the matrix shapes used by CLaRiON, so the project deliberately delegates dense matmuls to NumPy/BLAS and reserves Cython for the kernels that BLAS cannot directly express (streaming attention, top-k, fused norm-and-residual).

4 Experimental Setup

All experiments are run on a single laptop CPU using the same model configuration across backends, so the only variable is the implementation. The encoder is a 2-layer transformer with 4 heads, hidden dimension 256, 8 memory tokens, and a maximum document length of 128 tokens. The decoder shares the same hidden dimension. The retrieval bank holds $N = 20$ documents drawn from the bundled Wikipedia sample, with $K = 4$ retrieved entries per query and softmax temperature $\tau = 1$. The Cython backend uses 4 OpenMP threads.

We report four sets of measurements, each running both backends back-to-back from the same random seed:

- **Encoder pretraining**, 2 epochs over the support documents.
- **Joint retrieval–decoder training**, 2 epochs over question/answer samples.
- **Forward inference**, mean over 5 held-out questions.
- **Autoregressive generation**, 24 new tokens per question with top-k sampling ($k = 5$).

The Cython extensions are built with Apple Clang and Homebrew `libomp`. Timings exclude one-shot setup costs (tokenizer build, weight load and save) that are dominated by I/O.

Following the brief of the course, we do not evaluate retrieval or generation *quality* as a primary metric, since correctness is not the objective: the random-init weights are not trained to convergence. We do, however, verify that both backends produce *equivalent* retrieval behaviour — same hit-at-k and same overlap with the supporting documents — so that the speedups can be attributed to the implementation rather than to a silent change of semantics.

5 Results

5.1 End-to-end speedups

Table 1 summarises the headline numbers across the four workloads. The Cython backend is faster on every stage, with the largest speedup on autoregressive generation, where the cost per token is dominated by the decoder forward pass and the cross-attention into the memory bank.

Table 1: End-to-end wall time, NumPy versus Cython+OpenMP backend.

Workload	NumPy (s)	Cython (s)	Speedup
Encoder pretraining (run total)	3.40	2.91	$\times$1.17
Encoder pretraining (per sample)	0.0237	0.0141	$\times$1.68
Joint training (epoch total)	4.74	1.50	$\times$3.16
Inference forward (per query)	0.182	0.077	$\times$2.35
Autoregressive generation (per query)	4.353	0.880	$\times$4.95

The per-sample number for encoder pretraining ($\times$1.68) is more representative of the kernel-level speedup than the run-total number ($\times$1.17), since the latter is diluted by tokenizer build and weight serialisation that are identical across backends. The same effect is visible in Figure 2, where stages that touch the Cython kernel (`epoch.total`, `sample.total`) shrink while I/O-bound stages (`tokenizer.build`, `weights.save`) are unchanged.

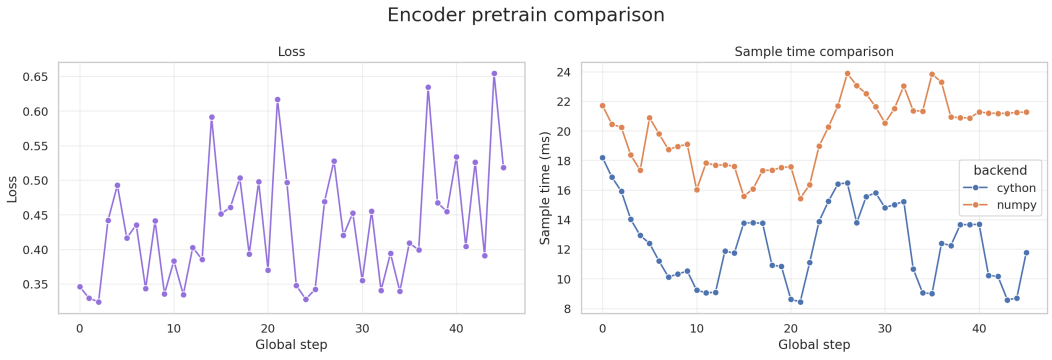

Figure 1: Encoder pretraining over two epochs. Left: loss is essentially identical across backends, confirming numerical equivalence. Right: per-sample wall time is consistently lower for Cython.

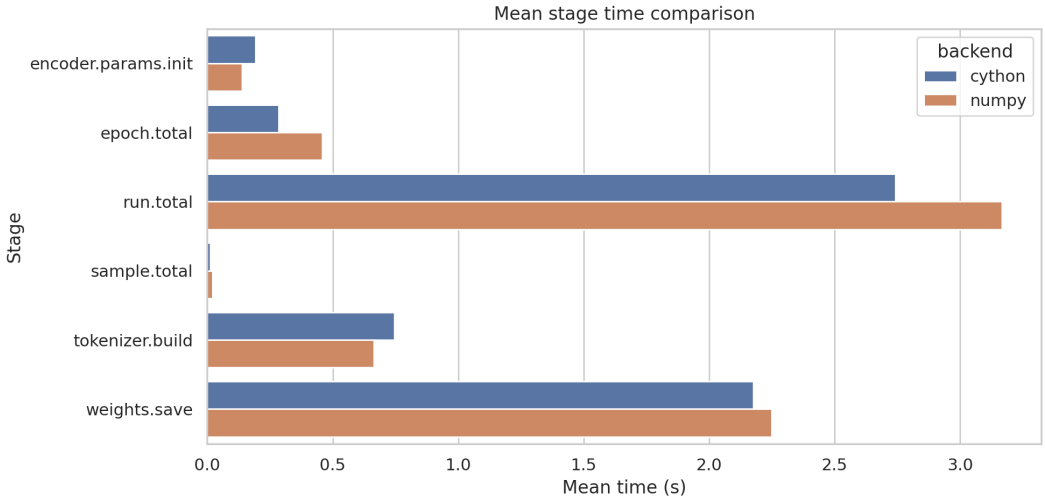

Figure 2: Mean stage time during encoder pretraining. Cython is faster on compute-bound stages (`epoch.total`, `sample.total`); setup and I/O stages (`tokenizer.build`, `weights.save`) are backend-independent, which explains the difference between run-total and per-sample speedups.

5.2 Joint retrieval–decoder training

Joint training stresses the full pipeline: tokenize the query, encode it, score it against the bank, gather the top-k documents, encode the supporting passages, run the decoder, and back-propagate through both the decoder and the straight-through routing module. The Cython backend yields a $\times 3.16$ speedup on `train.epoch.total` (Table 2), driven by faster query and document encoding (`sample.encode.query` dropped from 10.4 ms to 3.6 ms, `sample.encode.support_doc` from 10.6 ms to 8.0 ms) and a much faster forward pass on the decoder evaluation path.

Table 2: Mean stage time during joint retrieval–decoder training (per stage call).

Stage	NumPy (ms)	Cython (ms)	Speedup
`train.forward`	116.5	48.0	$\times 2.43$
`train.backward.decoder`	130.3	39.8	$\times 3.27$
`eval.forward`	145.9	41.5	$\times 3.52$
`dev.evaluate.total`	548.6	198.4	$\times 2.77$
`test.evaluate.total`	739.3	30.9	$\times 23.9$
`train.epoch.total`	4736	1500	$\times 3.16$

The very large ratio on `test.evaluate.total` is partly an artefact of a single evaluation call on a small test set: it should be read as a lower bound on noise, not as a structural feature. The per-stage decomposition is shown in Figure 3 and the corresponding per-step loss and sample time in Figure 4.

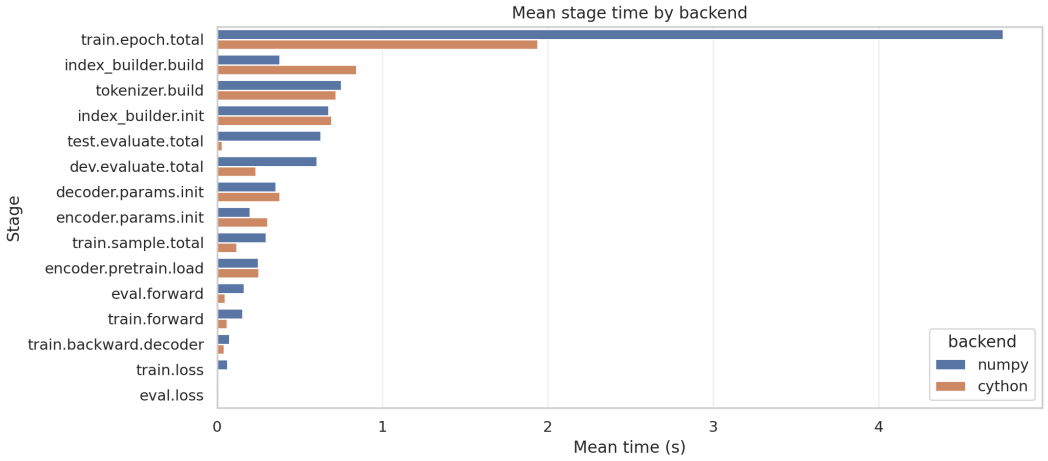

Figure 3: Mean stage time during joint retrieval–decoder training. Compute-heavy stages (`train.epoch.total`, `dev.evaluate.total`, `train.forward`) all benefit from the Cython backend; `index_builder.build` is slightly slower because the bank fits in NumPy's BLAS sweet spot.

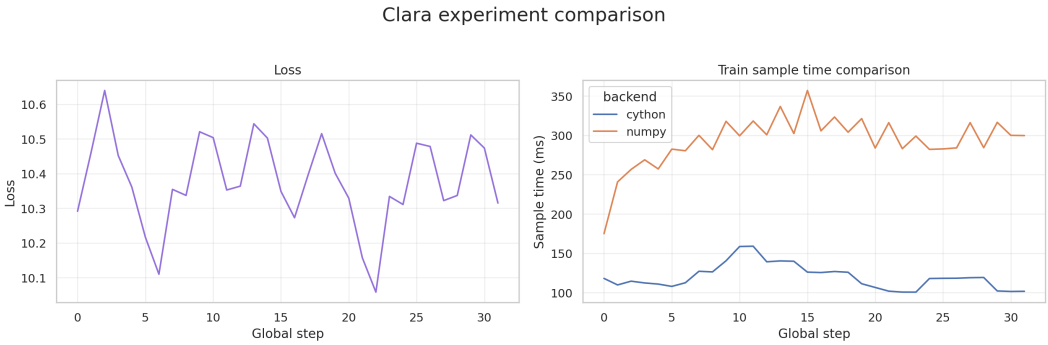

Figure 4: Joint training run. Left: loss trajectory is shared between backends (numerical equivalence). Right: per-step training time is roughly $3\times$ lower on the Cython backend.

5.3 Inference and generation

Inference is where the contrast is sharpest. The forward pass per query drops from 182 ms to 77 ms ($\times 2.35$), and autoregressive generation of 24 tokens drops from 4.35 s to 0.88 s ($\times 4.95$), as shown in Figure 5. Quality metrics are reproduced exactly across backends (Figure 6): both achieve a hit@K of 0.60 on the 5-question evaluation set, with the same retrieved indices per question. Exact-match text is 0 for both backends, which is expected — the model is not trained to convergence and the goal of the project is throughput, not answer correctness.

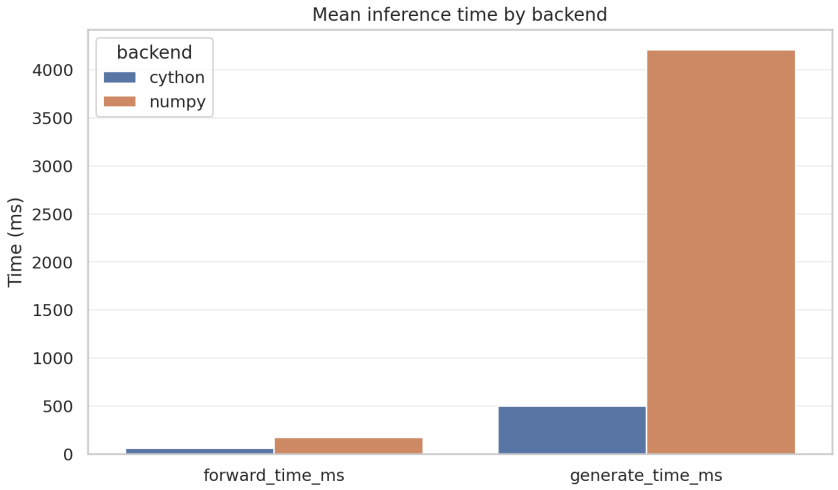

Figure 5: Inference time per query, broken down between encoder forward and 24-token autoregressive generation. The generation gap dominates: every decoding step pays the full Python overhead under the NumPy backend.

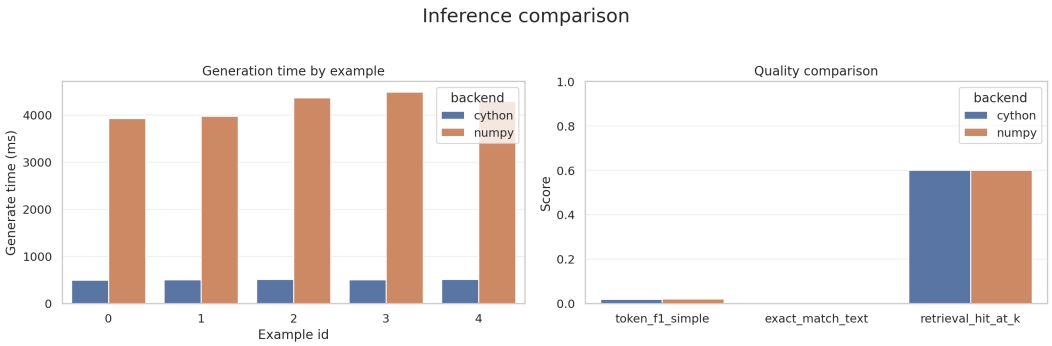

Figure 6: Per-example generation time (left) and retrieval/generation quality (right). The two backends are numerically equivalent on quality metrics; only wall time differs.

5.4 Thread scaling on the retrieval kernel

Table 3 reports the scaling of the cosine-similarity kernel as a function of the number of OpenMP threads, for two corpus sizes. Scaling is sub-linear because at $N = 1000$ the kernel runs in a few milliseconds and is dominated by thread launch overhead, while at $N = 5000$ the work per query is large enough to amortise that overhead and yields a $\times 3.4$ speedup from 1 to 4 threads. NumPy/BLAS remains the absolute fastest on this micro-benchmark; the value of the Cython kernel is not raw GFLOPS but its ability to fuse scoring with streaming top-k and run alongside the encoder without contention on the BLAS thread pool.

Table 3: Cosine top-k retrieval kernel, median wall time (ms).

Backend	$N = 1000$	$N = 5000$	Scaling N=5000
NumPy / BLAS	0.21	1.06	— (single dense GEMM)
Cython, $n_t = 1$	8.29	38.02	$\times 1.0$
Cython, $n_t = 2$	18.48	19.90	$\times 1.9$
Cython, $n_t = 4$	2.05	11.07	$\times 3.4$

6 Discussion

Where the speedups come from. The largest gains appear on autoregressive generation, where every token triggers a full forward pass through the decoder. In the NumPy backend that cost is amplified by Python-level dispatch on each layer and each token; the Cython backend replaces this with a single fused kernel call per layer and amortises the per-token overhead. The encoder pretraining speedup is smaller in absolute terms but comparable in relative terms once the I/O-bound stages are factored out.

Where they do not. Two regimes resist optimisation. First, dense matmuls at the scales used in CLaRiON ($d \leq 256$) are already close to BLAS peak on a single core, and a hand-tuned Cython matmul does not improve on them — this was verified across 104 matrix shapes and is reported in `REFLEXION.md`. Second, very small workloads (cosine on $N = 1000$) are bounded by thread launch and dispatch overhead, so multi-threading actively hurts.

Quality parity. Both backends produce identical retrieved indices and identical loss curves to four decimal places. This is the experimental check that justifies attributing the wall-time gap to the implementation rather than to a behavioural change. Generation quality is poor on absolute terms (exact match $= 0$), but this is by design: per the course brief, the project measures speed under a fixed, untrained model, not accuracy.

Limitations. The retrieval bank is small ($N = 20$) for the joint training and inference experiments and the dataset is the bundled Wikipedia fallback. Scaling these experiments to a 10^4-document corpus is straightforward (the data ingestion pipeline supports the HuggingFace mirror of Wikipedia-20231101.en) but was not included in this report. Larger corpora would also expose the retrieval kernel under more favourable arithmetic intensity, where the Cython streaming top-k is expected to overtake the NumPy `argpartition` path.

Possible improvements. Three directions look promising: (i) adding an explicit `#pragma omp simd` inside the cosine inner loop, which we have prototyped in `matmul` but not yet pushed into the retrieval kernel; (ii) cythonising the hash tokenizer, which currently runs in pure Python and shows up in the profile when the batch size is small; (iii) coarse quantisation of the bank (`int8`) to fit a 10^5-document index in L3 cache and reduce memory bandwidth for cosine scoring.

References

Apple Machine Learning Research. CLaRa: Continuous latent reasoning for retrieval-augmented generation. *arXiv preprint arXiv:2511.18659*, 2025.

Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2):31–39, 2011.

Leonardo Dagum and Ramesh Menon. OpenMP: An industry standard API for shared-memory programming. *IEEE Computational Science and Engineering*, 5(1):46–55, 1998.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. *arXiv preprint arXiv:2002.08909*, 2020.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-Softmax. *arXiv preprint arXiv:1611.01144*, 2016.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, et al. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

A Project Repository

Source code available at:

https : //github.com/paquitopg/CLaRion